# Learning to Probe Before Acting: A Multi-Turn RL Approach

**Abstract**

We study credit assignment for LLM agents that act through code over multiple turns. Standard outcome-reward RL samples one completion and broadcasts a single scalar advantage across every generated token: workable when the trajectory is one block of text, coarse when an early probe or draft changes the information available to later turns. Decisions whose effect passes through external state, tool outputs, and subsequent prompts then receive the wrong credit signal.

We propose a critic-free estimator that branches rollouts at turn boundaries. For each prompt the model samples several first-turn variants and several continuations under each variant; leaf rewards define a root Monte Carlo baseline for the prompt and a per-prefix baseline for each first-turn variant, so first-turn tokens are credited only when their prefix improves the expected reward of downstream continuations. Paired with token-summed clipped policy gradients, the estimator admits a per-turn loss decomposition that recovers the exact full-trajectory gradient while substantially reducing peak activation memory.

We instantiate the method on a two-turn code-policy task in which a first-turn probe can identify a hidden environment parameter the final policy needs, and lay out a staged ablation that separates the per-turn credit signal from branching, from multi-turn training, and from single-turn RL. Experiments are pending.

## 1 Introduction

Modern outcome-reward RL for LLMs samples a completion, scores it, and broadcasts a single advantage across every generated token. When the completion is one block of text, that approximation is at worst coarse: each token influences later tokens only by extending the prefix. When the trajectory contains environment probes, scratchpad programs, tool calls, and drafts whose effects reach later turns through external state, it can be wrong. An early turn that gathers information is rewarded only if the final answer happens to succeed; a final answer that succeeds despite an uninformative early turn rewards the early turn anyway. The trajectory's structure — which turns matter, and through what channel — is invisible to the gradient.

We study this in code-policy generation. The agent is given an environment interface, interacts for two chat turns, and emits a Python `policy(state)` function. Only this final policy is executed and scored; the deployed object is a program, not an LLM queried online. The first turn is free: scratchpad arithmetic, environment probes, draft policies. Useful first-turn behavior receives credit only through its effect on the second turn. The setting borrows the executable-artifact framing of FunSearch and AlphaEvolve and the broader motivation of AI-Scientist-style discovery loops [10, 11, 12], but isolates the credit-assignment problem inside one prompt rather than building a full discovery system.

The concrete instance is probe-and-adapt control. The environment hides an actuator parameter from the policy state. A first-turn program can command a short pulse, observe the response, and infer the parameter; a second-turn policy can rescale its controller accordingly. If the final

policy succeeds, a flat reward cannot say whether the probe was informative or whether the second-turn code happened to work without it.

We propose a critic-free Monte Carlo estimator for this situation. For each prompt the model samples a small tree: several first-turn variants, and several second-turn continuations below each variant. The mean reward over the whole tree estimates the prompt's root value; the mean below one variant estimates that prefix's value. Their difference advantages the first-turn tokens; the gap between a leaf reward and its prefix value advantages the second-turn tokens. The estimator pairs with a per-turn loss decomposition for token-summed clipped surrogates: because the loss is additive over disjoint token sets when all pieces share a global normalizer, one forward/backward per turn recovers the exact full-trajectory gradient at substantially reduced peak activation memory.

We instantiate the method on a custom control task in which a first-turn probe can reveal a hidden actuator parameter that the final policy needs. Experiments are pending.

The contributions are:

1. a critic-free turn-level Monte Carlo baseline with per-turn advantages for sparse-reward multi-turn RL;
2. an exact per-turn loss decomposition for token-summed clipped policy-gradient objectives;
3. a probe-and-adapt benchmark and an ablation design isolating the per-turn credit signal.

## 2 Related work

**Sparse-reward multi-turn LLM agents.** Recent work treats LLM agents as sequential decision-makers rather than single completions. ArCHer uses learned turn-level values for language agents [6]. RAGEN/StarPO studies trajectory-level reinforcement learning for interactive agents [4]. Agent Lightning casts arbitrary agent executions as MDP trajectories with a credit-assignment module [13]. Other work adds explicit turn-level rewards to multi-turn PPO or GRPO [14]. These approaches address the same failure mode: a final reward alone is too coarse to say which turn mattered. Our method differs by estimating turn-level baselines from final rewards alone, without a learned critic or manually designed intermediate reward.

**Monte Carlo credit assignment without critics.** GRPO replaces PPO's learned critic with group-relative Monte Carlo baselines [1]. Token-summed variants such as DAPO modify the normalization for long language-model completions [2]. These methods are critic-free, but a flat group baseline still assigns the same advantage to every token in a trajectory. VinePPO addresses a related issue by re-branching from intermediate reasoning states and estimating values with Monte Carlo continuations [3]. Our estimator is the turn-level analogue: the branch points are chat/tool boundaries, and the resulting scalar advantage is broadcast to the tokens of the corresponding turn.

**Executable discovery and code generation.** FunSearch, AlphaEvolve, and related systems show that LLMs can search over executable programs using automated evaluators [10, 11]. Work on automatic discovery of RL algorithms broadens this view to machine-designed update rules [15]. CodeRL, RLTF, and SWE-RL use execution or rule-based feedback to train code generators [16, 17, 18]. ReVeal is closer to our setting because it uses multi-turn code generation with tool feedback and dense per-turn rewards [19]. Our setting is different in that the first turn is valuable primarily as information gathering: it may reveal a hidden environment parameter that the final policy should exploit.

**Scientific agents.** AI-Scientist-style systems use LLMs to generate hypotheses, write and run code, analyze results, and draft papers [12]. These systems motivate our setting, but they operate at a higher level of abstraction. We study a smaller trainable component: how to assign credit inside a multi-turn, tool-mediated interaction when only the final executable artifact receives a reward.

## 3 Method

### 3.1 Setup

The agent is a language model $\pi_\theta$ over chat-template trajectories. The base is `Qwen3-4B-Instruct-2507` with frozen weights $W$; $\theta$ parameterizes a LoRA adapter $W' = W + AB^\top$. A task $\mathcal{T}$ specifies environment source code, $S$ scenarios $\{\sigma_i\}$ that vary the seed and hidden parameters, and a per-scenario reward. The scoring oracle runs a candidate policy body in a sandboxed subprocess against each scenario and returns $R(\text{code}; \mathcal{T}) = \sum_i r_i(\text{code}; \sigma_i)$. Validation failures, errors, and timeouts on $\sigma_i$ score the *error floor* $r_i = -200$, calibrated below any running-but-failing trajectory — otherwise errored rollouts ($r = 0$) outrank running-but-poor rollouts and the gradient rewards crashing.

### 3.2 Chat-template protocol

Each turn the model emits one fenced code block tagged `policy` or `main`. `policy` defines `def policy(state)`, which the tool executes across all scenarios and reports the summed reward as a user-role message. `main` is scratchpad or probe code run in a rehearsal sandbox; the tool returns truncated stdout but no final policy reward. The final turn must be `policy`; `main` or untagged final turns are floored. Malformed intermediate turns are likewise floored and surfaced as a textual rejection. A length-$N$ trajectory is $\tau = (s, y^{(1)}, m^{(1)}, \ldots, y^{(N-1)}, m^{(N-1)}, y^{(N)})$, with completions $y^{(n)}$ alternating with tool messages $m^{(n)}$, and $R(\tau) = R(\text{code in } y^{(N)})$ the only scalar reward. The remainder of the paper specializes to $N$=2: a single intermediate turn (draft policy or scratchpad) followed by a committed `policy`. Generalizing the rollout, advantages, and loss decomposition below to $N$>2 is straightforward (see §6).

### 3.3 Token-summed clipped surrogate

For trajectory $b$ at position $t$, with importance ratio $\rho_t^b = \pi_\theta / q$ and per-token advantage $A_t^b$,

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{b,t} \mathbf{1}\{\text{model token}\} \cdot \min\big(\rho_t^b A_t^b,\ \text{clip}(\rho_t^b,\ 1-\epsilon_{\text{lo}},\ 1+\epsilon_{\text{hi}}) A_t^b\big), \tag{1}$$

with $N_{\text{tok}}$ the count of model tokens in the global batch.[1] (1) is *additive over disjoint token subsets*: for any partition of model-token positions, the loss equals the sum of partial losses computed with the same $N_{\text{tok}}$. This is the property the per-turn decomposition (§3.6) exploits.

### 3.4 Branched rollout

For each prompt $s$, sample $G_1$ turn-1 variants in one `model.generate` call: $y_i^{(1)} \sim q(\cdot \mid s)$, $i = 1, \ldots, G_1$. For each $y_i^{(1)}$ execute the tool to obtain $m_i$, then sample $G_2$ continuations $y_{i,j}^{(2)} \sim q(\cdot \mid s, y_i^{(1)}, m_i)$ in one call. Score each leaf to obtain $\{R_{i,j}\}$. Generate-call cost per prompt is $1 + G_1$, versus $2G_1G_2$ for an unbranched rollout with the same number of leaves.

[1] This matches the DAPO and CISPO recipes; per-row averaging variants (original GRPO, BNPO, Dr.GRPO) divide by per-row token counts.

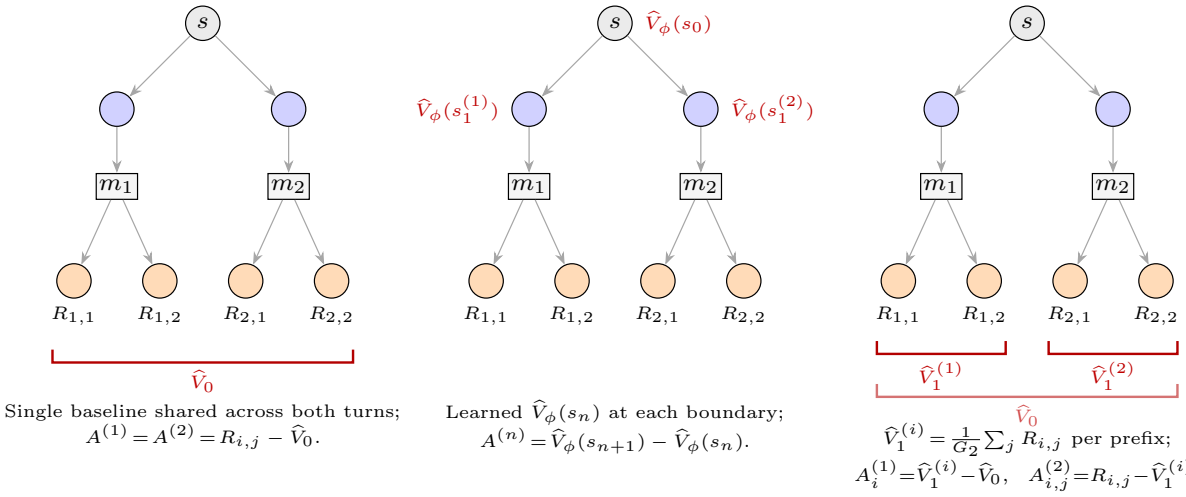



Figure 1: Comparison of credit-assignment mechanisms on a 2-turn branched rollout from prompt $s$ with $G_1$=$G_2$=2. All three panels show the same rollout structure — turn-1 fan-out $\{y_i^{(1)}\}$, tool message $m_i$, turn-2 fan-out $\{y_{i,j}^{(2)}\}$, leaf rewards $R_{i,j}$ — and differ only in how the per-token baseline is computed. **(a) Flat baseline** (StarPO / RAGEN [4], the N=2 generalization of RLOO/GRPO [1] from Figure 2 of [3]): one mean $\widehat{V}_0$ over all $G_1G_2$ leaves, broadcast to every model token of both turns. A poor turn-2 from a strong turn-1 inherits its prefix's above-average outcome. **(b) Per-turn critic** (ArCHer [6]): a learned value head $\widehat{V}_\phi(s_n)$ provides per-state baselines at each turn boundary, with the moving-target bias issues typical of bootstrapped value learning. **(c) Per-turn MC** (ours, Arm F): the conditional mean $\widehat{V}_1^{(i)} = \frac{1}{G_2}\sum_j R_{i,j}$ over the $G_2$ continuations of turn-1 variant $i$ baselines that variant's turn-1 tokens; $\widehat{V}_0$ baselines the root. Turn-2 tokens are scored against their own prefix's $\widehat{V}_1^{(i)}$, not the global mean. Group-relative within a turn, VinePPO-like across turns — i.e., VinePPO's intra-completion MC re-branching [3], applied at turn boundaries.

### 3.5 Per-turn MC value estimates and advantages

Two empirical values, both pure averages over the tree:

$$\widehat{V}_0 = \frac{1}{G_1 G_2}\sum_{i,j} R_{i,j}, \qquad \widehat{V}_1^{(i)} = \frac{1}{G_2}\sum_{j} R_{i,j}. \tag{2}$$

$\widehat{V}_0$ estimates the initial-state value; $\widehat{V}_1^{(i)}$ the value after committing to turn-1 variant $y_i^{(1)}$. Per-turn advantages, broadcast to every token of the corresponding turn:

$$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0 \qquad \text{(over } y_i^{(1)}\text{)}, \tag{3}$$

$$A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)} \qquad \text{(over } y_{i,j}^{(2)}\text{)}. \tag{4}$$

The turn-2 advantage uses the conditional mean given the turn-1 prefix rather than the global per-prompt mean: a poor turn-2 sample drawn from a strong turn-1 is recognized as poor instead of inheriting its prefix's above-average outcome. Tool-message tokens are masked via `tool_mask`; their advantage is zero. Figure 1 contrasts this scheme with the two closest critic-free / per-turn-credit baselines for multi-turn LLM RL.

### 3.6 Per-turn loss decomposition

The token-summed clipped surrogate (1) factors over any partition of the model-token positions into disjoint subsets that share the same global normalizer $N_{\text{tok}}$. Letting $\mathcal{I}_n$ index the model

tokens of turn $n$,

$$\mathcal{L} = \sum_{n=1}^{N} \mathcal{L}_{\mathcal{I}_n}, \qquad N_{\text{tok}} = \sum_{n=1}^{N} |\mathcal{I}_n|. \tag{5}$$

Per-row averaged variants (original GRPO, BNPO, Dr.GRPO) do *not* factor this way: each row carries its own denominator, so the partial sums do not combine into the single-row loss. The token-summed form is what makes the decomposition exact.

(5) is exploited to compute $\mathcal{L}$ in $N$ forward/backward passes rather than one. Pass $n$ forwards $[s \,|\, y^{(1)} \,|\, m^{(1)} \,|\, \ldots \,|\, y^{(n)}]$, computes $\mathcal{L}_{\mathcal{I}_n}$, backwards, and frees. By (5) the parameter gradients accumulated across the $N$ passes equal the single-pass gradient exactly.

**Why this saves memory.** The dominant activation cost is the LM-head logit tensor ($|V| =$ 151,936). A single forward+backward over the full trajectory holds the saved-for-backward LM-head tensors at all $\sum_n T_n$ model-token positions simultaneously; doing one forward+backward per turn holds at most $\max_n T_n$ positions at peak, and (5) guarantees the accumulated gradient is identical. This is gradient accumulation at the turn level within a trajectory — the saving is a factor of roughly $N$ for equal-length turns, $\approx 2\times$ at our $N$=2 settings. Appendix A works through the accounting.

**Restrictions.** The decomposition's exactness has one *conceptual* requirement and two *implementation* requirements. Conceptual: the token-summed normalizer of (1) — per-row averaged variants do not factor across token-position partitions, as discussed above. Implementation: $\beta = 0$ (a KL term to $\pi_{\text{ref}}$ would require per-segment slicing of reference log-probabilities) and `num_iterations`= 1 (so cached `old_per_token_logps` need no re-segmenting). The two implementation requirements can be lifted with extra bookkeeping at no theoretical cost (§6).

## 4 Experimental setup

**Environment.** We use a custom slosh-tank control task rather than a standard benchmark whose solution may already be familiar to the base model. A cart carries liquid represented by a damped pendulum mode. The policy observes $\{t, x, v, \theta, \omega, g, \text{steps_left}\}$ and outputs acceleration clipped to $[-1, 1]$ m/s$^2$. Success requires reaching the goal while also settling velocity, slosh angle, and slosh velocity. In the V1 task, the only hidden parameter is actuator gain $\kappa \in \{0.65, 0.80, 1.00\}$; all other parameters are fixed. Full dynamics are in Appendix B.

**Why it tests turn-level credit.** A first-turn probe can estimate $\kappa$ by commanding a short pulse and measuring $\Delta v/(a_{\text{cmd}}\Delta t)$. The final policy can then rescale its controller. This makes the first turn useful only through its effect on the final policy. Whether the gap between a blind controller and a $\kappa$-aware oracle is large enough to carry the study is settled with hand-written controllers before any model is run.

**Model and training.** We train LoRA rank $r = 16$ on attention projections of `Qwen3-4B-Instruct-2507`; base weights are frozen, bf16 is used, and gradient checkpointing is enabled. Unless stated otherwise, $G_1 = G_2 = 4$ and each turn is capped at 400 tokens. The loss is the token-summed clipped surrogate (1) with $\epsilon_{\text{lo}} = 0.2$, asymmetric upper clipping, $\beta = 0$, no critic, no GAE, and no context truncation across turns.

**Diagnostics.** We log $\widehat{V}_0$, the between-prefix and within-prefix reward standard deviations, their ratio, per-turn advantage scales, error-floor rate, and generate calls per prompt. The between/within ratio is also used diagnostically: values above one indicate that first-turn choice dominates continuation noise, where per-turn credit should help most.

## 5 Evaluating the per-turn credit assignment

The substantive empirical claim is the credit-assignment contribution: hierarchical per-turn advantages (3)–(4) should outperform a flat trajectory-level advantage broadcast across both turns. Table 1 defines the six-arm ablation. The key comparisons are E vs. D for the rollout tree and F vs. E for the per-turn baseline.

| Arm | Rollout structure | Advantage | What it isolates over the previous arm |
|---|---|---|---|
| A — Frozen, one-shot | $N$=1 inference, no training | — | Base model's prior on the code-policy interface |
| B — Frozen, multi-turn prompted | $N$=2 inference, no training | — | Does the two-turn protocol help at inference time? (Voyager-/FunSearch-style frozen-LLM use) |
| C — Single-turn GRPO | $G$ flat $N$=1 rollouts, trained | Standard GRPO advantage | Does RL fine-tuning help at all? |
| D — Flat StarPO | $G$ independent $N$=2 trajectories, trained | $A(\tau) = R(\tau) - \bar{R}$, broadcast to all tokens | Does multi-turn training help over single-turn? (RAGEN-style) |
| E — Branched, flat | $G_1 \times G_2$ tree, trained | $A_{i,j} = R_{i,j} - \bar{R}$ on all tokens | Does the tree alone help (more leaves per generate call), independent of credit? |
| F — Branched, hierarchical (ours) | $G_1 \times G_2$ tree, trained | $A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0$ on $y_i^{(1)}$; $A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)}$ on $y_{i,j}^{(2)}$ | The per-turn baseline contribution |

Table 1: Ablation arms, ordered from weakest baseline (top) to ours (bottom). Trained arms (C–F) match leaf count $G = G_1 G_2 = 16$ so they see the same number of scored trajectories per step. D uses $2G$ `generate` calls per prompt; F uses $1 + G_1$. Report both equal-rollouts and equal-wall-clock comparisons. Frozen arms (A, B) use `Qwen3-4B-Instruct-2507` (§3.1) with no gradient updates, and serve as training-free references analogous to Voyager- or FunSearch-style frozen-LLM uses of the code-policy interface.

**Primary metric.** Held-out four-condition success rate on a stress distribution (more extreme actuator-gain values than train, i.e. $\kappa$ further from 1) at the best checkpoint, reported as median and IQR over $\geq 3$ seeds per arm, plotted against gradient steps and against wall-clock. Statistical test: paired Wilcoxon over per-seed best-checkpoint goal rates; report effect size, not just $p$. The minimum convincing comparison is F > D at matched leaves.

**Diagnostic checks.** We track two mechanism checks. First, when turn 1 emits a `policy`, its reward should improve faster under F than under flat multi-turn training. Second, prompts with high iteration-0 signal ratio $\widehat{\sigma}(\widehat{V}_1^{(i)})/\overline{\widehat{\sigma}(R_{i,\cdot})}$ should show the largest F–D gain; otherwise the proposed credit-assignment explanation is weak.

**Failure modes.** If F does not beat D, the hierarchical baseline is not useful in this domain. If F $\approx$ E while both beat D, the benefit comes from branching rather than turn-level credit. If C–F do not beat A/B, the protocol is better framed as prompting rather than RL training.

## 6 Limitations and future work

The implementation and experiments use $N = 2$. Extending to more turns is conceptually direct: each boundary value becomes the conditional Monte Carlo mean over the subtree below it, and

the loss decomposition applies one partial loss per turn. The cost is bookkeeping and additional forward/backward passes, not a new estimator.

The method also assumes deterministic rewards given the seed. In stochastic environments, the same estimator applies but $G_2$ must grow to separate prefix quality from environment noise. The current implementation uses $\beta = 0$ and `num_iterations`$= 1$; KL penalties and cached old log-probabilities can be supported by storing the same quantities per turn segment.

A natural memory/variance extension is token subsampling within a turn. Since each turn uses one scalar advantage broadcast across its tokens, subsampling tokens is an unbiased estimator of the same token-summed gradient when $N_{\text{tok}}$ counts only kept positions. We leave uniform and asymmetric subsampling variants to future work.

## 7 Conclusion

We present a critic-free, turn-level credit-assignment scheme for sparse-reward multi-turn RL. The method branches rollouts at chat-turn boundaries, uses the resulting Monte Carlo conditional means as per-turn baselines, and pairs with a per-turn loss decomposition that recovers the exact full-trajectory gradient one turn at a time.

The experiments that test this — frozen one- and two-turn references, single-turn RL, flat trajectory-level multi-turn optimization, and branched optimization with per-turn baselines — are pending. The estimator extends naturally to more turns and to stochastic rewards (Section 6); whether the per-turn signal closes the gap between a blind and an informed final policy is the headline empirical claim under test.

## A Memory accounting and the trailing-window LM-head

The loss only needs logits whose successor token was emitted by the model; prompt and tool-message logits do not contribute. HF `transformers` exposes this through `logits_to_keep`: an integer keeps the last $K$ positions, while an index tensor selects arbitrary positions.

This reduces unnecessary logits, but a single forward/backward still holds all selected LM-head outputs until backward completes. With gradient checkpointing, transformer-body activations are comparatively small; the expensive saved tensor is the LM-head output of shape $(B, K, |V|)$. Therefore, a full multi-turn backward over all model tokens scales with $K = \sum_n T_n$.

The per-turn decomposition serializes these positions. Turn $n$ forwards the prefix through $y^{(n)}$, computes the partial token-summed loss for that turn using the global $N_{\text{tok}}$, backpropagates, and frees the graph. By additivity of (1), the accumulated gradient equals the single-pass gradient, while the peak LM-head tensor scales with $\max_n T_n$.

| Regime | Positions through LM head | Logit tensor |
|---|---|---|
| Single pass, no `logits_to_keep` | $T_{\text{full}} = 1350$ | $\approx 13.1$ GB |
| Single pass, selected model tokens | $\sum_n T_n = 800$ | $\approx 7.8$ GB |
| Two turn-wise passes | 400 per pass, freed between | $\approx 3.9$ GB |

Table 2: Arithmetic LM-head logit sizes for $B = 16$, $|V| \approx 152{,}000$, and 4-byte logits. Total GPU memory also includes weights, optimizer state, body activations, KV cache, and allocator overhead.

The decomposition is not a new `logits_to_keep` trick; all regimes use the same primitive. The non-trivial point is that token-summed normalization makes the per-turn gradients sum exactly to the full-trajectory gradient. Per-row-averaged variants would require either a different normalizer or an approximation.

**Forward-side temporaries change the binding constraint.** Table 2 accounts for the LM-head logits tensor as it appears in the saved-for-backward computation graph. At production sizing on a 24 GB card, the binding allocation is not the gradient at backward but the forward itself. The softmax-stability convention in TRL holds logits in fp32 rather than bf16, doubling the byte budget over a naive estimate. Worse, the temperature scaling `logits = logits / self.temperature` is not in place: a same-shape fresh tensor is allocated before the original is freed. At the divide step *two* $(B, T, |V|)$ fp32 tensors are simultaneously live, $\approx 7.8$ GiB at our production sizing. Stacked on model weights, the just-emitted turn-$n$ KV cache from generation that has not yet been reclaimed by the allocator, and the previous turn's backward residue, this allocation crosses the 22 GiB usable budget and OOMs at step 0.

The per-turn loss decomposition addresses peak *gradient* memory but does not shrink the forward logits tensor. The remedy is row-wise chunking of the per-turn pass: split the $B$ trajectories into sub-batches of size $b \ll B$, forward each sub-batch separately, backpropagate, and accumulate. Because the token-summed loss (1) is additive over disjoint row subsets sharing the same global normalizer $N_{\text{tok}}$, the chunked gradient is bit-equivalent to the single-pass gradient. The forward-time logits tensor shrinks linearly from $(B, T_n, |V|)$ to $(b, T_n, |V|)$, and the divide-step peak scales correspondingly. At $b = 4$, $B = 16$, $T = 400$ the divide-step peak drops from $\approx 7.8$ GiB to $\approx 2$ GiB; the cost is $B/b$ additional LM-head kernel launches per turn (single-digit-percent wall-clock at our generation-dominated sizing). We use $b = 4$ on 24 GB L4 and run un-chunked on $\geq 40$ GB cards.

## B Slosh-tank environment

A one-dimensional cart carries liquid represented by one damped pendulum mode. The policy commands horizontal acceleration clipped to $[-1, 1]$ m/s$^2$ and observes $\{t, x, v, \theta, \omega, g, \text{steps_left}\}$. Hidden parameters can include actuator gain $\kappa$, delay $D$, deadband $b$, slosh length $L$, and damping $c$; V1 varies only $\kappa$.

The semi-implicit dynamics use $\Delta t = 0.06$ s:

$$\begin{aligned} a_{\text{true}} &= \kappa \,\text{deadband}(a_{\text{cmd}[t-D]}), \\ v \mathrel{+}= \Delta t\, a_{\text{true}}, &\qquad x \mathrel{+}= \Delta t\, v, \\ \dot{\omega} = -\frac{g}{L}\theta - c\omega - a_{\text{true}}, &\qquad \dot{\theta} = \omega. \end{aligned} \tag{6}$$

Success requires $|x - g| < \epsilon_x$, $|v| < \epsilon_v$, $|\theta| < \epsilon_\theta$, and $|\omega| < \epsilon_\omega$ simultaneously, preventing controllers that reach the goal without settling the liquid.

**Why actuator gain is the V1 unknown.** With goal distance $D_g \approx 1.8$ m and $a_{\max} = 1$ m/s$^2$, the minimum bang-bang transit time is $T_{\text{tr}} = 2\sqrt{D_g / a_{\max}} = 2.68$ s. The episode time gives slack $\tau = T_{\text{ep}}/T_{\text{tr}} = 2.35$. Thus low $\kappa$ does not mainly cause transit failure; it slows near-goal settling because all corrective actions are attenuated. This creates a smooth blind-vs-oracle gap while keeping the probe interpretable: for a short pulse near $\theta \approx \omega \approx 0$, $\kappa \approx \Delta v / (a_{\text{cmd}} \Delta t)$.

**Confirmed V1 configuration.**

| Parameter | Symbol | V1 value | Status |
|---|---|---|---|
| Episode steps | $N_{\text{steps}}$ | 105 ($\tau = 2.35$) | fixed |
| Actuator gain | $\kappa$ | $\{0.65, 0.80, 1.00\}$ | varied |
| Action delay | $D$ | 0 | fixed |
| Deadband | $b$ | 0 | fixed |
| Slosh length | $L$ | 0.70 m | fixed |
| Damping | $c$ | 0.06 s$^{-1}$ | fixed |

The two-turn protocol is useful only if a probe-aware controller can close some of this gap. The blind controller sets the no-probe baseline, the oracle sets the upper bound, and a hand-written pulse-probe controller is the achievable probe-aware reference.

## References


[1] Z. Shao et al. *DeepSeekMath*. arXiv:2402.03300, 2024.

[2] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476, 2025.

[3] A. Kazemnejad et al. *VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment*. arXiv:2410.01679, 2024.

[4] Z. Wang et al. *RAGEN: Training Agents by Reinforcing Reasoning*. 2025.

[5] Z. Liu et al. *GEM: Reinforcement Learning for Generalist Models*. 2025.

[6] Y. Zhou et al. *ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL*. ICML, 2024.

[7] P. Wang et al. *Math-Shepherd*. ACL, 2024.

[8] L. Luo et al. *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592, 2024.

[9] G. Wang et al. *Voyager*. TMLR, 2024.

[10] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024.

[11] A. Novikov et al. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. 2025.

[12] C. Lu et al. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. arXiv:2408.06292, 2024.

[13] X. Luo et al. *Agent Lightning: Train ANY AI Agents with Reinforcement Learning*. arXiv:2508.03680, 2025.

[14] W. Zeng et al. *Reinforcing Multi-Turn Reasoning in LLM Agents via Turn-Level Reward Design*. arXiv:2505.11821, 2025.

[15] J. Oh et al. *Discovering State-of-the-art Reinforcement Learning Algorithms*. Nature, 2025.

[16] H. Le et al. *CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning*. NeurIPS, 2022.

[17] J. Liu et al. *RLTF: Reinforcement Learning from Unit Test Feedback*. 2023.

[18] Y. Wei et al. *SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution*. 2025.

[19] M. Jin et al. *ReVeal: Reinforcement Learning for Verifiable Multi-Turn Code Generation*. 2025.